

Bridge Day 14 INSTRUCTOR KEY

Lists, Length, Indexes, Mutation, and List Loops

Learning sequence: Predict - Trace - Explain - Test - Interpret

Thursday, July 23, 2026 • 50 points • longitudinal template 07242026_v1

Name		UIN		Section	
------	--	-----	--	---------	--

Daily target and independence boundary

Choose index evidence when stored values must be read, changed, summarized, and located.

Allowed tools	Prior tools plus lists, len, append, indexed reads, and indexed assignment
Support supplied	A starting list and required final-state summaries are supplied.
Student decides	Loop form, list mutation, summary state, and first-largest tracking.
Scaffold level	Specification only; no planning prompts or variable inventory.

Evidence and scoring

Evidence	Points	Secure work shows
Integrated trace	10	intermediate state, condition results, exact output, and meaning
Diagnosis and repair	8	actual, intended, cause, repair, and a discriminating test
Complete program and tests	32	coherent executable logic, required output, assumptions, and defended tests

Task 1. Integrated trace - 10 points

Trace index state, list values, an appended evidence list, and an accumulator in one ledger.

```
values = [6, 3, 9, 4]
total = 0
low_positions = []

for index in range(len(values)):
    if values[index] < 5:
        low_positions.append(index)
    else:
        total = total + values[index]

print(total, low_positions, values[len(values) - 1])
```

Your task

1. Give the complete index-by-index ledger and exact output, distinguishing each index from its stored value.

Answer 1. Index 0 stores 6, so total becomes 6 and low_positions stays empty. Index 1 stores 3, so index 1 is appended. Index 2 stores 9, so total becomes 15. Index 3 stores 4, so index 3 is appended. The last valid index is $\text{len}(\text{values}) - 1 = 3$, which stores 4. Exact output: 15 [1, 3] 4.

Task 2. Diagnose and repair - 8 points

The intent is to count every negative value, including the first and last positions.

```
values = [-2, 4, -1, 6, -3]
negative_count = 0

for index in range(1, len(values) - 1):
    if values[index] < 0:
        negative_count = negative_count + 1

print(negative_count)
```

Your task

1. Give actual versus intended result, identify both skipped boundaries, repair the loop, and name one compact test that protects both ends.

Answer 1. The loop visits indexes 1, 2, and 3, so it counts only -1 and prints 1. The intended count is 3 because indexes 0, 2, and 4 are negative. Repair with `range(len(values))`. Protective test: `[-1, 2, -3]` must produce 2; it fails if either first or last index is skipped.

Task 3. Construct a complete program - 32 points

Program specification

- Use readings = [12, -3, 7, 20, 7]. The list is nonempty.
- Loop by index and replace every negative value with 0.
- Using final values, compute valid_total and the number of values at or above 10.
- Find the index of the first largest final value without using max or index.
- Print the final list, valid_total, high_count, and largest_index on separate lines.

Program workspace**Model program**

```
readings = [12, -3, 7, 20, 7]
valid_total = 0
high_count = 0
largest = readings[0]
largest_index = 0

for index in range(len(readings)):
    if readings[index] < 0:
        readings[index] = 0

    value = readings[index]
    valid_total = valid_total + value
    if value >= 10:
        high_count = high_count + 1
    if value > largest:
        largest = value
        largest_index = index

print(readings)
print(valid_total)
print(high_count)
print(largest_index)
```

Reasoning: Mutation occurs before the summaries, so total, threshold count, and largest state all describe the final list.

Reasoning: The strict greater-than comparison preserves the first largest index if a tie occurs. An index loop is required because the program both writes to positions and reports a position.

Task 3 continued. Test and defend - included in 32 points

Continue code if needed.

Expected tests and results

1. The supplied list becomes [12, 0, 7, 20, 7] and prints total 46, high_count 2, and largest_index 3.
2. Tie test [9, -1, 9] becomes [9, 0, 9] and must keep largest_index 0.
3. Boundary test [-2, 4, -1] becomes [0, 4, 0] and protects first/last mutation.

Required tests, expected results, and brief defense

Scoring guidance: 5 initialization; 5 complete indexed traversal; 5 negative mutation; 5 final-state total/count; 6 first-largest tracking; 3 output; 3 tests/explanation.

Bridge Day 14 INSTRUCTOR KEY

VIP Evidence Handoff

Learning sequence: Predict - Trace - Explain - Test - Interpret

Contract 07a04826c975 • longitudinal template 07242026_v1

Why engineers use this page

Begin with the notebook's required input/conversion/f-string reconnect, then carry index, value, and final-list evidence into reverse traversal and one- or two-condition filters.

Decision boundary: Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

Construct readiness before independent work

New authoring tools: string traversal, list traversal, list index read, list index write, string index read, slice expression, membership test, parallel list indexing, compound filter

Carried authoring tools: assignment, print call, arithmetic expression, input call, int conversion, float conversion, f string, comparison expression, boolean operator, if elif else, for loop, range call, append call, len call

Supplied-code exposure only: none

An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

Notebook cells and task constructs

Activity	Work	Stable cells	Required authoring constructs
18.25	Reverse Values	18-25-work / 18-25-transfer / 18-25-cold	append call, for loop, range call
18.26	Values At Or Above Cutoff	18-26-work / 18-26-transfer / 18-26-cold	append call, comparison expression, for loop
18.27	Feasible Workstations	18-27-work / 18-27-transfer / 18-27-cold	append call, boolean operator, f string, float conversion, for loop, input call, int conversion, range call

Readiness evidence

1. State which mapped activity requires indexes and which activities can preserve order by traversing values.

Model response: 18.25 requires reverse index traversal. Activity 18.26 can traverse values directly. Activity 18.27 must use one shared index across the supplied parallel lists.

2. For Activity 18.27, write the two Boolean constraints separately before combining them with 'and'.

Model response: The exact constraints come from the supplied setup-time and capacity lists. Both comparisons must be true before names[index] is appended.

Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** _____ **My help trigger:** _____

Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

Evidence field	Record
Prediction	A specific expected state or output before execution.
Observed Result	The actual state or output, including a value or exact text.
Revision Reason	The defect or mismatch and the change that addresses it.
Engineering Meaning	How the evidence changes or supports the engineering decision.
Units Or Not Applicable	The physical unit, or the evidence type when no unit applies.
Assumption	One condition accepted as true for this model or rule.
Model Limit	One situation the result does not justify or predict.
Next Test	A concrete boundary, invalid, or alternate case with an expected result.
Help Trigger	The exact unresolved point that would trigger a targeted question.
Minutes Used	Approximate minutes from first prediction through revision.

Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.